LÝ THUYẾT ĐỒ THỊ

knguyen – vhung – vanh

Đồ thị là gì?

Đồ thị là cấu trúc toán học gồm:

$$G = (V, E)$$

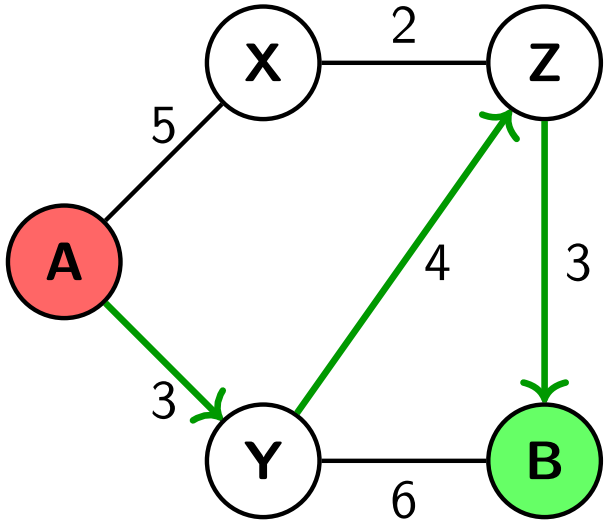
- V : tập các đỉnh
- E : tập các cạnh nối các đỉnh

Google Maps – Tìm đường đi ngắn nhất

Bài toán

Tìm đường đi ngắn nhất từ **A** đến **B**

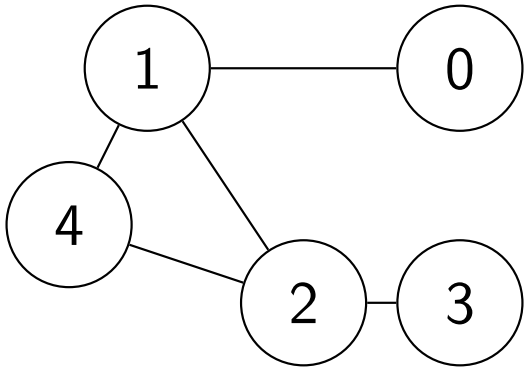
- Các giao lộ → **Đỉnh (Vertices)**
- Các con đường → **Cạnh (Edges)**
- Thời gian / Khoảng cách → **Trọng số**



Thuật toán gợi ý của Hưng:

Sử dụng **Dijkstra** để tìm đường đi ngắn nhất

Ví dụ đồ thị

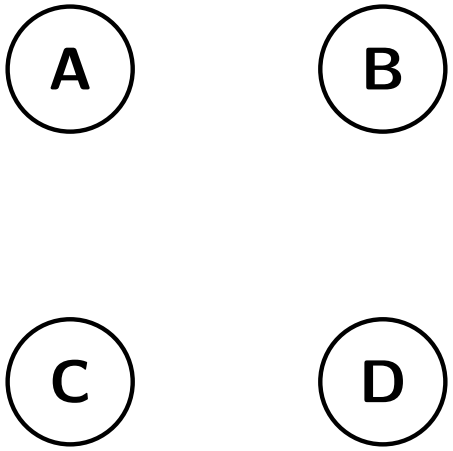


Đỉnh (Vertex)

Khái niệm

Đỉnh (vertex) là các điểm cơ bản cấu tạo nên đồ thị. Mỗi đỉnh thường biểu diễn một đối tượng.

- Ký hiệu: $V = \{A, B, C, D\}$
- Ví dụ: mỗi đỉnh là một thành phố, người, hoặc máy tính

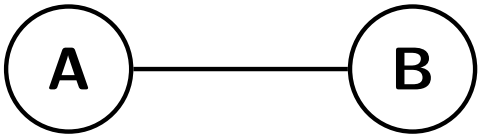


Cạnh (Edge)

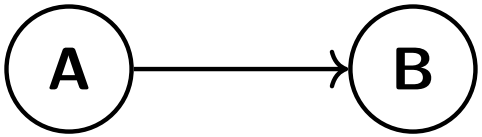
Khái niệm

Cạnh (edge) là đoạn nối giữa hai đỉnh, biểu diễn mối quan hệ giữa chúng.

- Đồ thị vô hướng: cạnh không có chiều
- Đồ thị có hướng: cạnh có mũi tên
- Đa đồ thị: có nhiều cạnh giữa 2 đỉnh



Vô hướng



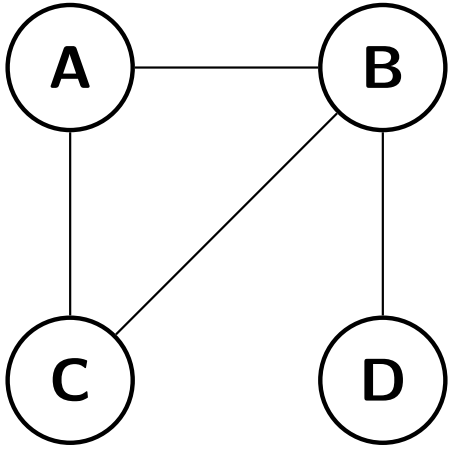
Có hướng

Bậc của đỉnh (Degree)

Khái niệm

Bậc của một đỉnh là số cạnh nối với đỉnh đó.
Với $\deg(\cdot)$ là hàm tính bậc của đỉnh, ta có:

- $\deg(A) = 2$
- $\deg(B) = 3$
- $\deg(C) = 2$
- $\deg(D) = 1$

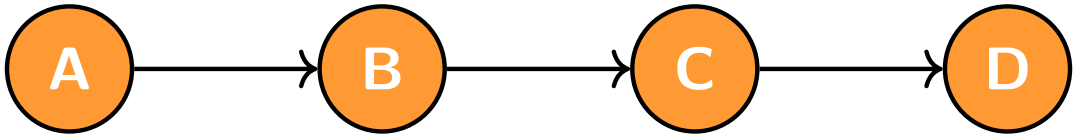


Đường đi (Path)

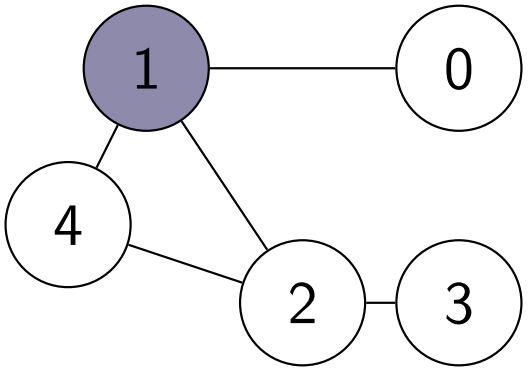
Khái niệm

Đường đi là dãy các đỉnh mà giữa các đỉnh liên tiếp đều có cạnh nối.

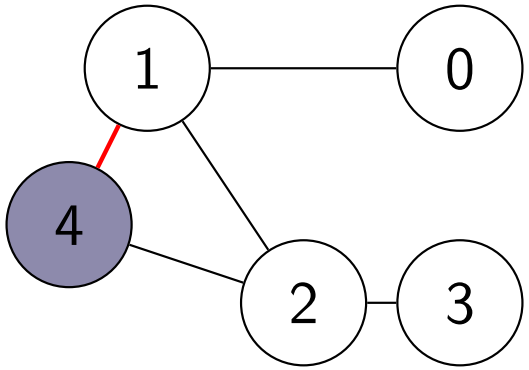
- Ví dụ: $A \to B \to C \to D$
- Độ dài đường đi = số cạnh đi qua



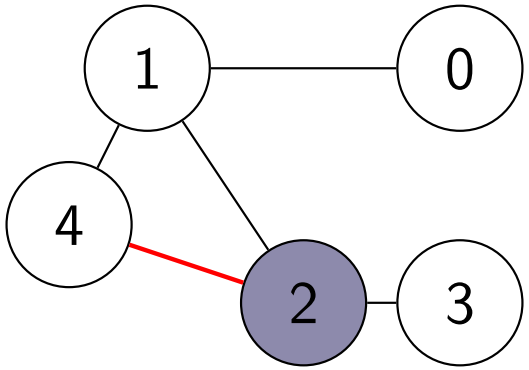
Bước 1: Bắt đầu



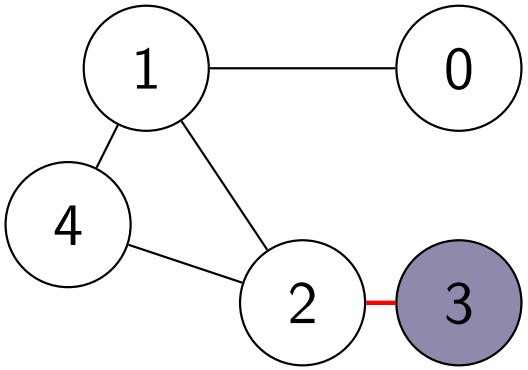
Bước 2: Di chuyển đến đỉnh 4



Bước 3: Di chuyển đến đỉnh 2



Bước 4: Di chuyển đến đỉnh 3



Biểu diễn: Danh sách kề (Adjacency List)

Cấu trúc: Mỗi đỉnh lưu một danh sách các đỉnh láng giềng.

- **Đỉnh 1:** [4, 0, 2]
- **Đỉnh 2:** [1, 3, 4]
- **Đỉnh 3:** [2]

Ưu điểm:

- **Tiết kiệm bộ nhớ:** Chỉ tốn $O(V + E)$.
- **Duyệt nhanh:** Duyệt các đỉnh kề mất $O(deg(u))$.
- Phù hợp cho đồ thị lớn, đồ thị thưa.

Biểu diễn: Danh sách kề

Danh sách kề sử dụng mảng vector để lưu trữ các đỉnh láng giềng.

```
int main() {
    int n, m; // số đỉnh, số cạnh
    cin >> n >> m;
    for(int i=0; i<m; i++){
        int u, v; cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u); // Đồ thị vô hướng
    }
    return 0;
}
```

Biểu diễn: Danh sách cạnh (Edge List)

- Lưu trữ danh sách các cặp đỉnh (u, v) tạo thành cạnh.
- Rất hiệu quả khi thuật toán chỉ cần duyệt qua từng cạnh một.
  Đặc điểm:
  - Dễ cài đặt, tốn ít bộ nhớ.
  - Phù hợp với các thuật toán chỉ quan tâm đến các cạnh (như Kruskal, Bellman-Ford).
  - Không hiệu quả khi cần tìm các đỉnh kề của một đỉnh cụ thể.

Ví dụ:

STT	Đỉnh u	Đỉnh v
1	1	4
2	1	0
3	1	2
4	2	3
5	4	2

Biểu diễn: Danh sách cạnh

Danh sách cạnh lưu trữ đồ thị dưới dạng tập hợp các cặp đỉnh.

```
struct Edge {
    int u, v;
};
vector<Edge> edgeList;

// Thêm cạnh
edgeList.push_back({u, v});
```

Biểu diễn: Ma trận kề (Adjacency Matrix)

Ma trận A kích thước $N \times N$:

- $A[i][j] = 1$: Nếu có cạnh nối i và j.
- $A[i][j] = 0$: Nếu không có.

Đồ thị 3 đỉnh:

$$A = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}$$

(Dòng 1 nối với đỉnh 2, Dòng 2 nối 1 và 3, v.v.)

Nhược điểm: Tốn bộ nhớ $O(N^2)$, không tối ưu với đồ thị có hàng chục ngàn đỉnh.

Biểu diễn: Ma trận kề

```
// Ma trận kích thước tối đa 100x100
int adj[100][100];
int main() {
    int n, m; // số đỉnh, số cạnh
    cin >> n >> m;

    for(int i = 0; i < m; i++) {
        int u, v; cin >> u >> v;
        // Đồ thị vô hướng: đánh dấu 1 ở cả 2 chiều
        adj[u][v] = 1;
        adj[v][u] = 1;
    }
    return 0;
}
```

Thuật toán DFS (Depth-First Search)

- **Nguyên lý:** "Đi sâu nhất có thể"trước khi quay lui (Backtracking).
- **Cấu trúc dữ liệu:** Sử dụng **Ngăn xếp (Stack)** hoặc **Đệ quy** (chính là Stack của hệ thống).
- **Các bước:**
  1. Thăm đỉnh u, đánh dấu u là đã thăm (visited).
  2. Với mỗi đỉnh v kề với u:
     - Nếu v chưa được thăm, thực hiện gọi đệ quy `DFS(v)`.

Lưu ý: DFS rất dễ cài đặt nhờ đệ quy, nhưng cần cẩn thận với đồ thị quá lớn vì có thể gây tràn Stack (Stack Overflow).

Cài đặt DFS với C++ (Đệ quy)

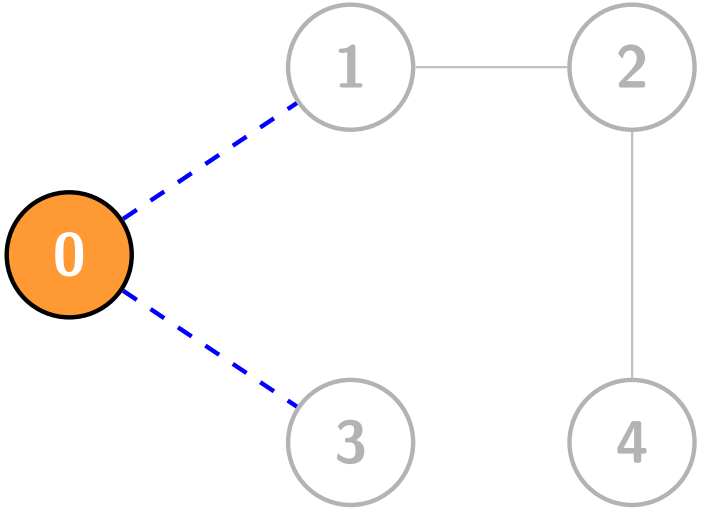
```
vector<int> adj[100]; // Danh sách kề
bool visited[100];    // Mảng đánh dấu đã thăm

void DFS(int u) {
    visited[u] = true;      // Đánh dấu u đã thăm
    // Duyệt qua các đỉnh kề v
    for(int v : adj[u]) {
        if(!visited[v]) {
            DFS(v);         // Gọi đệ quy cho v
        }
    }
}
```

Ghi chú: visited[] đóng vai trò cực kỳ quan trọng để ngăn chương trình lặp vô tận (nếu đồ thị có chu trình).

4. BFS: Tìm kiếm theo chiều rộng

- **Ý Tưởng:** Duyệt theo từng "lớp"vết dầu loang. Thăm hết tất cả các đỉnh kề trực tiếp rồi mới tiến ra xa hơn.
- **Đặc điểm:** Luôn tìm được đường đi ngắn nhất trong đồ thị không trọng số.
- **Độ phức tạp:** Thời gian $O(V + E)$.

Mô phỏng từng bước BFS

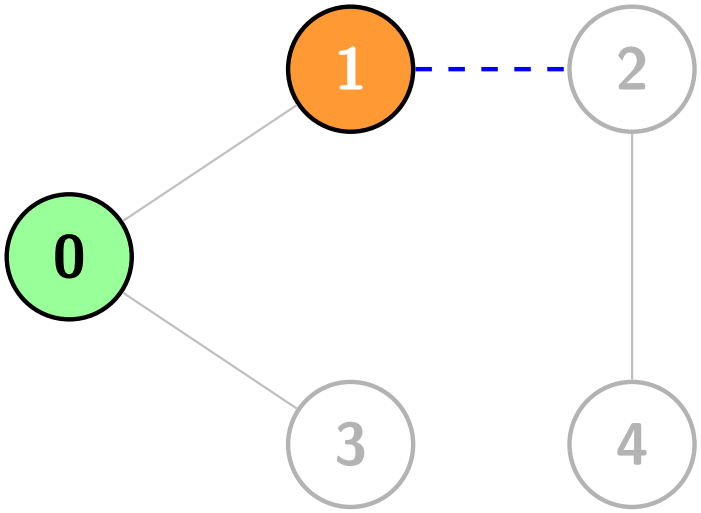


Hàng đợi Queue:

1	3

B1:
-Thăm 0
-Đưa đỉnh kề {1, 3} vào Queue.

Mô phỏng từng bước BFS

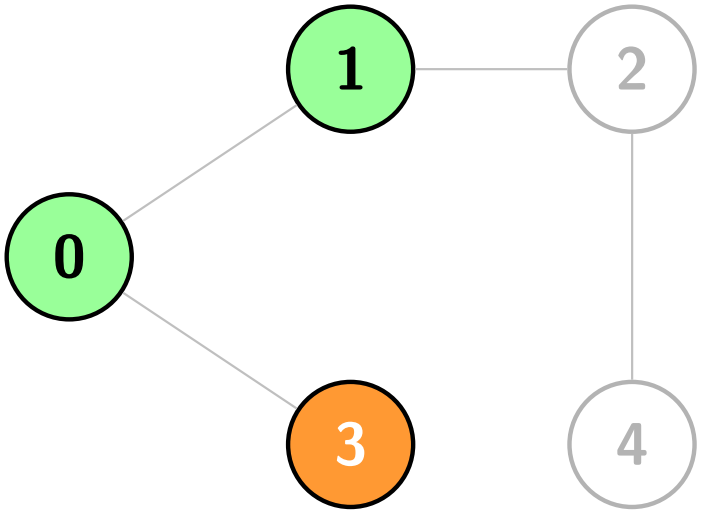


Hàng đợi Queue:

3	2

B2:
-Lấy 1 ra thăm.
-Thêm đỉnh kề 2 vào Queue.

Mô phỏng từng bước BFS

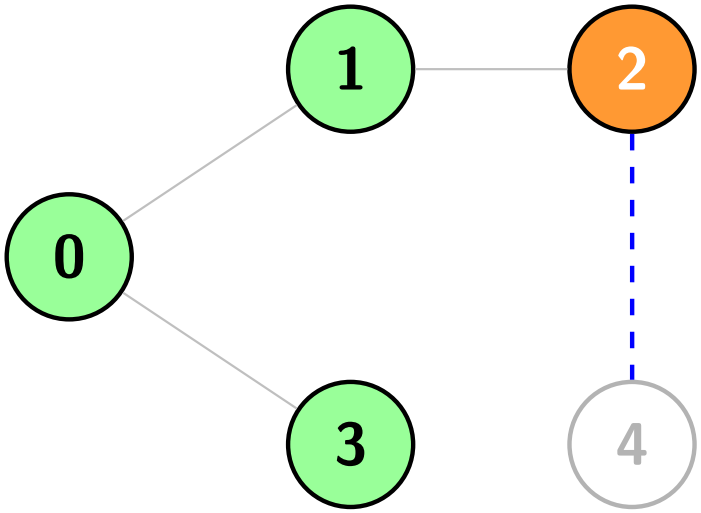


Hàng đợi Queue:

2

B3:
-Lấy 3 ra thăm.
-Đỉnh 0 đã được duyệt.

Mô phỏng từng bước BFS

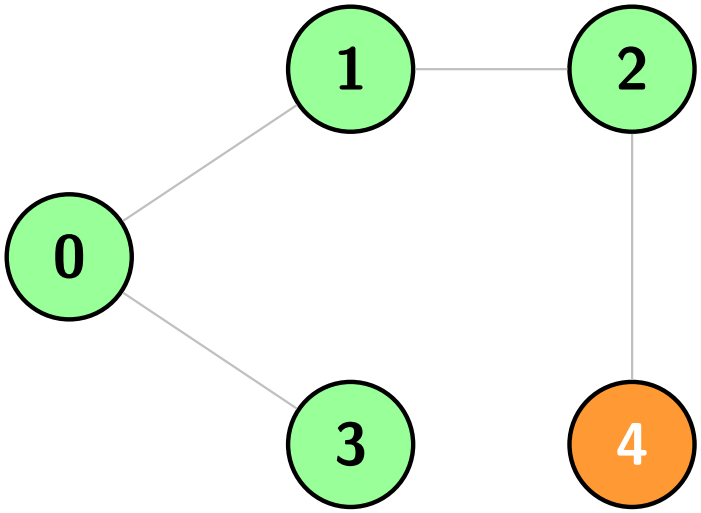


Hàng đợi Queue:

4

B4:
-Lấy 2 ra thăm.
-Thêm đỉnh kề 4 vào Queue.

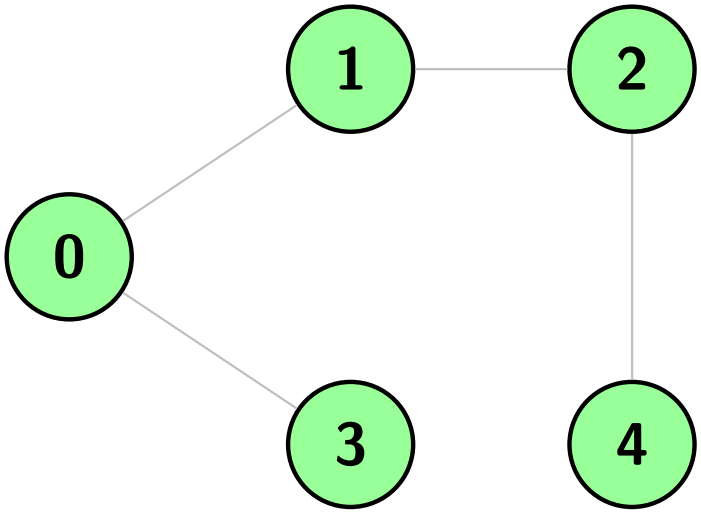
Mô phỏng từng bước BFS



Hàng đợi Queue:

B5:
-Lấy 4 ra thăm.
-Hoàn thành.

Mô phỏng từng bước BFS



Hàng đợi Queue:

Kết quả:
$0 \to 1 \to 3 \to 2 \to 4.$

Mã nguồn BFS (C++)

```
void bfs(int start_node) {
    queue<int> q;
    q.push(start_node);
    visited[start_node] = true;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        cout << u << " ";
        for (int v : adj[u]) {
            if (!visited[v]) {
                visited[v] = true;
                q.push(v);
            }
        }
    }
}
```

Ứng dụng của đồ thị

- Google Maps (Tìm đường đi ngắn nhất)
- Mạng xã hội (Mối quan hệ)
- Internet (Liên kết trang web)
- Game AI (Di chuyển nhân vật)

CẢM ƠN